Test 24h – Développeur Backend Node.js

Objectif : Livrer une API ultra-optimisée, scalable et documentée, avec gestion de la concurrence et des erreurs.

Contexte

Vous devez créer un service de génération de documents (CERFA, conventions) capable de traiter des lots de 1000 documents en parallèle, avec résilience et monitoring.

Consignes

1. API Core (8h)

- Créer une API Node.js/TypeScript avec Express.

- Endpoints :

  - POST /api/documents/batch : reçoit un tableau de 1000 userIds, lance la génération asynchrone, retourne un batchId.

  - GET /api/documents/batch/:batchId : retourne le statut du batch (pending, processing, completed, failed) et la liste des documents générés.

  - GET /api/documents/:documentId : retourne le PDF généré.

- Architecture :

  - Queue avec Bull (Redis) pour traiter les documents en parallèle (concurrency configurable).

  - Worker séparé (ou process cluster) pour la génération PDF.

  - Gestion des erreurs avec retry (3 tentatives, backoff exponentiel).

  - Stockage MongoDB : collection batches, collection documents (avec GridFS pour les PDF).

2. Performances (6h)

- Optimiser la génération PDF :

  - Utiliser worker threads pour la génération parallèle.

  - Mettre en cache les templates PDF (compile une fois, générer multiples).

  - Streaming des PDF vers la base sans charger en mémoire.

- Benchmark :

  - Écrire un script de test qui lance un batch de 1000 documents.

  - Mesurer : temps total, CPU, mémoire, nombre de documents/seconde.

  - Fournir un rapport détaillé avec courbes.

3. Résilience (4h)

- Implémenter :

  - Circuit breaker pour les appels externes (DocuSign simulé).

  - Health checks : /health (DB, Redis, queue).

  - Graceful shutdown (SIGTERM) pour terminer les jobs en cours.

- Gérer les cas extrêmes :

  - MongoDB indisponible.

  - Redis down (fallback en mémoire).

  - Timeout sur génération PDF (5s max).

4. Observabilité (4h)

- Logs structurés (JSON) avec corrélation (batchId, documentId).

- Métriques Prometheus :

  - documents_generated_total.

  - batch_processing_duration_seconds.

  - queue_size.

- Exposer un endpoint GET /metrics au format Prometheus.

- Dashboard simple (Grafana ou texte) avec les métriques clés.

5. Documentation (2h)

- Swagger/OpenAPI complet avec exemples.

- README incluant :
  - Architecture diagram (sequence pour batch processing).
  - Justification des choix techniques (pourquoi Bull, pourquoi GridFS).
  - Instructions de déploiement (Docker compose avec Redis, MongoDB).

Livrables

- Repository GitHub.
- Docker-compose.yml (Node.js, Redis, MongoDB).
- Script de benchmark avec rapport.
- Collection Postman ou curl commands.
- Lien vers l'API déployée (Render ou autre).

Critères d'évaluation

1. Fonctionnalité : Toutes les consignes sont-elles respectées ?
2. Architecture : Choix techniques justifiés, scalabilité, séparation des responsabilités.
3. Performance : Temps de réponse, utilisation des ressources.
4. Résilience : Gestion des erreurs, retries, fallbacks.
5. Tests : Couverture, qualité des cas testés.
6. Documentation : README clair, diagrammes, instructions.
7. Qualité du code : TypeScript strict, ESLint, conventions.
8. Sécurité : Validation, rate limiting, gestion des secrets.
9. Délivrabilité : L'application tourne en un docker-compose up.